Operating Systems – Midterm Examination
April 19, 2011

Total: 100 points

1. Hardware devices signal interrupts when important events occur (e.g., hard-disk data are available):
   (a) (5 points) Show the advantage of interrupt-based operations over busy-waiting.
   (b) (10 points) Describe the process of hardware interrupt handling. You need to mention device controller, interrupt-request line, state saving, interrupt vector, interrupt-specific handler.
   (c) (5 points) Explain the benefit of using interrupt vector.
2. (5 points) Explain the difference between multiprogramming and timesharing.
3. (15 points) Invoking system calls is tedious, so application programmers prefer using API functions to access the services provided by operating systems. Describe how an API function invokes a system call on behavior of a programmer. You need to mention software interrupt, dual-mode, system call number (index), and table of code pointer.
4. (10 points) Describe the pros and cons of layered operating system structure.
5. Show the state of a process when the process:
   (a) (3 points) is selected by a long-term scheduler
   (b) (3 points) is selected by a short-term scheduler
   (c) (3 points) invokes a blocking system call
   (d) (3 points) completes a blocking system call
   (e) (3 points) is interrupted by an interrupt
6. (20 points) Complete the following program which calculates $\sum_{n=1}^{N} n!$. The value N is provided by users in the command line, i.e., N = atoi(argv[1]):

   Hint: The size of an integer is 4 bytes. The program needs to create N child processes - one for each factorial calculation (i.e., $n!$). The child processes save the results in a shared integer memory and the parent process prints the final outcome. To prevent inconsistent results, parent needs to wait for each child process.

```
int main(int argc, char *argv[])
{
    int N = atoi(argv[1]);
    pid_t pid;
    int segment_id;
    int *shared_memory;

    // allocate and attach a shared memory segment


    for(          ) // create child processes
    {


        if(          ) {
            fprintf(stderr, "Fork Failed");
            exit(-1);

        } else if(          ) { // child part
            // do factorial calculation and save it
            // then detach shared memory and exit


        } else { // parent part

        }
    }

    printf("the result is %d\n",*shared_memory);

    // detach and remove the shared memory segment


    return 0;

}
```

這樣的variable 是child 可用的？（在哪宣布？ ex. global）

7. Multithread model:
   (a) (5 points) What is the problem of many-to-many thread model that scheduler activation tries to resolve?

block 時 ex. I/O.

   (b) (10 points) Describe the operation of scheduler activation.

Sh Scheduler table.

IPC_PMID